

Android-Only App Handbook (Kotlin + Jetpack)

A beginner field guide for Python developers

KotlinGuide

2026-08-07

- 1 Where do I look? (Fast map)
- 2 Chapter 1: Setup check (5 minutes)
- 3 Chapter 2: Project map + Gradle basics
 - 3.1 2.1 Typical structure
 - 3.2 2.2 Files you will touch most
 - 3.3 2.3 Gradle in beginner terms
- 4 Chapter 3: Python → Kotlin bridge
 - 4.1 3.1 Variables, types, null safety
 - 4.2 3.2 Conditions and loops
 - 4.3 3.3 Functions
 - 4.4 3.4 Classes + data classes
 - 4.5 3.5 Collections + lambdas
 - 4.6 3.6 Exceptions
- 5 Chapter 4: Android fundamentals
 - 5.1 4.1 Activity lifecycle (single-activity apps still use this)
 - 5.2 4.2 Core concepts
 - 5.3 4.3 Configuration changes
- 6 Chapter 5: Compose UI essentials (Material 3)
 - 6.1 5.1 Composables
 - 6.2 5.2 Modifiers, layout, text, buttons, input
 - 6.3 5.3 Lists and icons/images
 - 6.4 5.4 Theme + previews
 - 6.5 5.5 Accessibility + adaptive basics
- 7 Chapter 6: State + recomposition
 - 7.1 6.1 remember and mutableStateOf
 - 7.2 6.2 State hoisting
 - 7.3 6.3 Unidirectional data flow (UDF)
- 8 Chapter 7: Architecture without buzzword overload
 - 8.1 7.1 UI layer
 - 8.2 7.2 ViewModel
 - 8.3 7.3 Repository
 - 8.4 7.4 Coroutines and Flow (beginner framing)
- 9 Chapter 8: Navigation Compose
- 10 Chapter 9: Persistence (DataStore + Room)
 - 10.1 9.1 DataStore for preferences
 - 10.2 9.2 Room for structured data
- 11 Chapter 10: Networking (Retrofit + safe secrets)
 - 11.1 10.1 Basics
 - 11.2 10.2 Retrofit + Kotlin serialization
 - 11.3 10.3 Loading/error state pattern
 - 11.4 10.4 Secret safety checklist
- 12 Chapter 11: Background work with WorkManager
- 13 Chapter 12: Runtime permissions + privacy/security
- 14 Chapter 13: Debugging + troubleshooting tree
 - 14.1 13.1 Practical tools
 - 14.2 13.2 Common Gradle/build problems
 - 14.3 13.3 Troubleshooting decision tree
- 15 Chapter 14: Testing basics
 - 15.1 14.1 What to test first
 - 15.2 14.2 Local unit test vs UI test
- 16 Chapter 15: Release basics (signing, versioning, Play Store)
- 17 Chapter 16: Cohesive sample app (Task Tiny)
 - 17.1 16.1 Feature outline
 - 17.2 16.2 Core pieces
 - 17.2.1 Entity + DAO
 - 17.2.2 Repository
 - 17.2.3 ViewModel
 - 17.2.4 Navigation + screens

- 17.3 16.3 Why this sample matters
- 18 Troubleshooting quick section
- 19 Quick references (cheat sheets)
 - 19.1 Kotlin mini-cheat
 - 19.2 Compose state mini-cheat
 - 19.3 Coroutine + Flow mini-cheat
 - 19.4 Room mini-cheat
- 20 Glossary
- 21 Index (A-Z)
- 22 References (official docs first)

How to use this handbook: Start with the **Where do I look? map** below. Then jump to a chapter using the table of contents.

1 Where do I look? (Fast map)

If you need to...	Go to
Check Android Studio and devices quickly	Chapter 1: Setup check
Understand project files and Gradle	Chapter 2: Project map + Gradle basics
Translate Python thinking to Kotlin	Chapter 3: Python → Kotlin bridge
Learn app lifecycle, manifest, resources	Chapter 4: Android fundamentals
Build UI with Compose + Material 3	Chapter 5: Compose UI essentials
Understand state and recomposition	Chapter 6: State + recomposition
Structure code with ViewModel/repository	Chapter 7: Architecture without buzzword overload
Navigate between screens	Chapter 8: Navigation Compose
Save preferences and local data	Chapter 9: Persistence (DataStore + Room)
Call APIs safely	Chapter 10: Networking
Run background jobs	Chapter 11: WorkManager
Handle permissions and privacy	Chapter 12: Runtime permissions + privacy/security
Debug build/app issues fast	Chapter 13: Debugging + troubleshooting tree
Test the right things	Chapter 14: Testing basics
Prepare release + Play Store	Chapter 15: Release basics
See a full mini app flow	Chapter 16: Cohesive sample app
Look up terms quickly	Glossary
Find topics alphabetically	Index
Quick copy-paste reminders	Quick references

2 Chapter 1: Setup check (5 minutes)

Android Studio recent stable version installed
 Android SDK + Platform Tools installed
 Emulator image (x86_64 or arm64) downloaded
 USB debugging enabled on physical Android device (optional)
 adb devices shows your emulator/device

Tip: If your emulator is slow, use an x86_64 system image with hardware acceleration.

Quick health checks:

```
adb devices
```

If a device is missing:

- 1. Restart ADB: adb kill-server && adb start-server
- 2. Replug cable / authorize USB debugging again
- 3. For Windows, verify OEM USB driver

3 Chapter 2: Project map + Gradle basics

A modern Compose app usually uses a **single activity** and many composable screens.

3.1 2.1 Typical structure

```
app/  
  src/main/  
    AndroidManifest.xml  
    java/... (or kotlin/...)  
      ui/  
      data/  
      domain/ (optional)  
    res/  
      drawable/  
      values/  
      mipmap/  
    build.gradle.kts  
  build.gradle.kts (project)  
  settings.gradle.kts  
  gradle/libs.versions.toml
```

3.2 2.2 Files you will touch most

File	Why it matters
AndroidManifest.xml	Declares app components + permissions
app/build.gradle.kts	App dependencies + Android build settings
gradle/libs.versions.toml	Central version catalog
MainActivity.kt	App entry activity
ui/...	Compose screens/components
res/values/strings.xml	User-visible text

3.3 2.3 Gradle in beginner terms

- **Gradle** = build system (downloads libraries, compiles, packages APK/AAB).
- **Plugin** = extends Gradle (Android plugin, Kotlin plugin).
- **Dependency** = external library (Compose, Room, Retrofit).
- **Sync** = Android Studio refreshes project after build file changes.

⚠ **Common mistake:** editing dependency versions in multiple files. Prefer one source in `libs.versions.toml`.

4 Chapter 3: Python → Kotlin bridge

➔ **Python comparison callout** included in every section.

4.1 3.1 Variables, types, null safety

```
val name: String = "Elsa"    // immutable (like conventionally constant)
val score: Int = 0           // mutable
val nickname: String? = null // nullable type
```

Python vs Kotlin:

- Python: dynamically typed (name = "Elsa")
- Kotlin: statically typed; nullability is explicit (String vs String?)

```
val len = nickname?.length ?: 0
```

4.2 3.2 Conditions and loops

```
if (score > 10) {
    println("High")
} else {
    println("Low")
}

for (i in 0 until 3) println(i)
for (item in listOf("a", "b")) println(item)
while (score < 5) score++
```

4.3 3.3 Functions

```
fun greet(name: String, excited: Boolean = false): String {
    return if (excited) "Hi, $name!" else "Hi, $name"
}
```

4.4 3.4 Classes + data classes

```
class Counter {
    var value = 0
    fun inc() { value++ }
}

data class Task(val id: Long, val title: String, val done: Boolean)
```

data class auto-generates equals/hashCode/toString/copy.

4.5 3.5 Collections + lambdas

```
val tasks = listOf(
    Task(1, "Read", false),
    Task(2, "Code", true)
)

val openTitles = tasks
    .filter { !it.done }
    .map { it.title }
```

4.6 3.6 Exceptions

```
try {
    val n = "12a".toInt()
} catch (e: NumberFormatException) {
    println("Bad number")
}
```

```
}
```

△ Kotlin has checked-like clarity through types (Result, nullable types), but Java-style checked exceptions are not required.

5 Chapter 4: Android fundamentals

5.1 4.1 Activity lifecycle (single-activity apps still use this)

```
onCreate -> onStart -> onResume -> (running)
        <- onPause <- onStop <- onDestroy
```

- onCreate: initialize once.
- onStart/onResume: app visible/interactive.
- onPause/onStop: release expensive resources.

5.2 4.2 Core concepts

Term	Meaning
Activity	Android app entry window (host for Compose UI)
Manifest	App metadata, permissions, launch activity
Resource	Externalized values/assets (strings, colors, drawables)
Context	Handle to app/system services
Configuration change	Runtime environment change (rotation, font scale, locale)

5.3 4.3 Configuration changes

Compose state in plain remember is lost on activity recreation. Use:

- rememberSaveable for simple UI state
- ViewModel for screen/business state

6 Chapter 5: Compose UI essentials (Material 3)

6.1 5.1 Composables

```
@Composable
fun GreetingCard(name: String) {
    Text(text = "Hello, $name")
}
```

A composable is a function that describes UI.

6.2 5.2 Modifiers, layout, text, buttons, input

```
@Composable
fun NameEditor(name: String, onNameChange: (String) -> Unit, onSave: () -> Unit) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        Text("Profile", style = MaterialTheme.typography.headlineSmall)
    }
}
```

```

        OutlinedTextField(value = name, onValueChange = onNameChange, label = { Text("Name") })
        Button(onClick = onSave) { Text("Save") }
    }
}

```

6.3 5.3 Lists and icons/images

```

@Composable
fun TaskList(tasks: List<Task>) {
    LazyColumn {
        items(tasks, key = { it.id }) { task ->
            ListItem(
                headlineContent = { Text(task.title) },
                leadingContent = {
                    Icon(
                        imageVector = if (task.done) Icons.Default.CheckCircle else
Icons.Default.RadioButtonUnchecked,
                        contentDescription = if (task.done) "Done" else "Open"
                    )
                }
            )
        }
    }
}

```

6.4 5.4 Theme + previews

```

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    MaterialTheme {
        GreetingCard("Android")
    }
}

```

6.5 5.5 Accessibility + adaptive basics

- Provide meaningful contentDescription for important icons/images.
- Use minimum touch target ~48dp.
- Test large font scale and dark mode.
- Avoid fixed widths when possible; use Modifier.weight, responsive layouts.

Never rely on color alone: pair color with icon/text labels.

7 Chapter 6: State + recomposition

7.1 6.1 remember and mutableStateOf

```

@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) }
    Button(onClick = { count++ }) {
        Text("Count: $count")
    }
}

```

When count changes, Compose recomposes affected UI.

7.2 6.2 State hoisting

Keep state in parent, pass value + callbacks down.

```
@Composable
fun CounterHost() {
    var count by rememberSaveable { mutableStateOf(0) }
    CounterView(count = count, onInc = { count++ })
}

@Composable
fun CounterView(count: Int, onInc: () -> Unit) {
    Button(onClick = onInc) { Text("Count: $count") }
}
```

7.3 6.3 Unidirectional data flow (UDF)

User Action -> Event -> ViewModel updates State -> UI redraws from State

This avoids "mystery mutations" from many directions.

8 Chapter 7: Architecture without buzzword overload

Simple mental model:

UI (Compose) <-> ViewModel <-> Repository <-> Data sources (Room / network / DataStore)

8.1 7.1 UI layer

- Renders immutable UI state.
- Sends user events upward.

8.2 7.2 ViewModel

- Holds screen state across config changes.
- Launches coroutines in viewModelScope.

```
data class TasksUiState(
    val items: List<Task> = emptyList(),
    val isLoading: Boolean = false,
    val errorMessage: String? = null
)
```

8.3 7.3 Repository

- Single place coordinating local + remote data.
- Exposes Flow<T> for observable streams.

8.4 7.4 Coroutines and Flow (beginner framing)

- **Coroutine**: lightweight async task.
- **Flow**: stream of values over time.

```
val uiState: StateFlow<TasksUiState>
```

You will hear "MVVM". Here it only means this practical split: **View (Compose)**, **ViewModel (state logic)**, **Model/data (repository + sources)**.

9 Chapter 8: Navigation Compose

Use NavHost with route strings or typed routes.

```
@Composable
fun AppNav() {
    val navController = rememberNavController()
    NavHost(navController, startDestination = "list") {
        composable("list") {
            ListScreen(onOpenDetail = { id -> navController.navigate("detail/$id") })
        }
        composable("detail/{taskId}") { backStackEntry ->
            val id = backStackEntry.arguments?.getString("taskId")?.toLongOrNull() ?:
return@composable
            DetailScreen(taskId = id, onBack = { navController.popBackStack() })
        }
    }
}
```

Guidelines:

- Pass small IDs as args, not huge objects.
- Keep route constants in one place.

10 Chapter 9: Persistence (DataStore + Room)

10.1 9.1 DataStore for preferences

Use for key-value settings (theme, sort order).

```
val Context.dataStore by preferencesDataStore(name = "settings")
```

10.2 9.2 Room for structured data

Use for relational local data with SQL-backed entities.

```
@Entity(tableName = "tasks")
data class TaskEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val title: String,
    val done: Boolean
)

@Dao
interface TaskDao {
    @Query("SELECT * FROM tasks ORDER BY id DESC")
    fun observeAll(): Flow<List<TaskEntity>>

    @Insert
    suspend fun insert(task: TaskEntity)
}
```

11 Chapter 10: Networking (Retrofit + safe secrets)

11.1 10.1 Basics

- Add `<uses-permission android:name="android.permission.INTERNET" />` in manifest.
- API call lifecycle: request → loading → success/error.

11.2 10.2 Retrofit + Kotlin serialization

```
@Serializable
data class QuotedDto(val id: String, val text: String)
```

```
interface QuoteApi {
    @GET("quotes/random")
    suspend fun randomQuote(): QuoteDto
}
```

11.3 10.3 Loading/error state pattern

```
sealed interface LoadState<out T> {
    data object Loading : LoadState<Nothing>
    data class Success<T>(val data: T) : LoadState<T>
    data class Error(val message: String) : LoadState<Nothing>
}
```

11.4 10.4 Secret safety checklist

- Never hardcode API keys in source.
- Never commit secrets to Git.
- Use secure backend when possible.
- For local dev, inject placeholders via `local.properties` / CI secrets.
- Restrict keys server-side (domain/IP quotas, scopes, rotation).

12 Chapter 11: Background work with WorkManager

Use WorkManager for **deferrable, guaranteed** background tasks (sync, upload retry).

Do **not** use WorkManager for:

- immediate UI actions needing instant completion
- exact-alarm precision tasks

```
class SyncWorker(appContext: Context, params: WorkerParameters) :
    CoroutineWorker(appContext, params) {
    override suspend fun doWork(): Result {
        return try {
            // sync
            Result.success()
        } catch (e: Exception) {
            Result.retry()
        }
    }
}
```

13 Chapter 12: Runtime permissions + privacy/security

Runtime permissions (camera/location/etc.) should be requested at the moment of need.

```
val launcher = rememberLauncherForActivityResult(
    contract = ActivityResultContracts.RequestPermission()
) { granted ->
    // update state
}
```

Privacy/security mini-checklist:

- Ask only necessary permissions.
- Explain why before prompt.
- Handle denial gracefully.
- Minimize data collection.
- Use HTTPS.
- Avoid logging PII/secrets.

14 Chapter 13: Debugging + troubleshooting tree

14.1 13.1 Practical tools

- **Logcat:** filtered runtime logs.
- **Breakpoints:** pause and inspect values.
- **Layout Inspector:** inspect Compose hierarchy.
- **Build output:** first failure is often root cause.

14.2 13.2 Common Gradle/build problems

Symptom	Likely cause	First fix
Dependency not found	wrong repo/version	sync, verify version catalog
Kotlin/Compose mismatch	incompatible compiler/plugin	align Kotlin + Compose versions
Duplicate class	conflicting transitive deps	inspect dependency tree
Manifest merge failed	conflicting metadata	inspect merged manifest

14.3 13.3 Troubleshooting decision tree

Build fails?

└ Yes -> Read FIRST red error line

└ Dependency error -> check version catalog + sync

└ Kotlin/Compose compile error -> inspect changed file + imports

└ Manifest error -> inspect merged manifest + permissions/activities

└ Still stuck -> Invalidate caches + restart + clean build

└ No (runs but broken)

└ UI wrong -> use Preview + Layout Inspector

└ Logic wrong -> add breakpoint + inspect state transitions

└ Data wrong -> verify repository flow + Room query + API payload

15 Chapter 14: Testing basics

15.1 14.1 What to test first

1. ViewModel state transitions (loading/success/error).
2. Repository logic with fake data sources.
3. Compose UI critical interactions (button click, text shown).

15.2 14.2 Local unit test vs UI test

Test type	Runs on	Good for
Local unit test	JVM	business logic, ViewModel, mapping
Instrumented/Compose UI test	device/emulator	UI behavior, semantics

Compose test sketch:

```
composeTestRule.onNodeWithText("Save").performClick()
composeTestRule.onNodeWithText("Saved").assertExists()
```

16 Chapter 15: Release basics (signing, versioning, Play Store)

High-level flow:

1. Update versionCode (int, always increases) and versionName (human-readable).
2. Create signed release build (AAB preferred).
3. Verify release notes + privacy declarations.
4. Upload to Play Console (internal testing track first).

⚠ Keep signing keys secure and backed up. Losing app signing access is painful.

17 Chapter 16: Cohesive sample app (Task Tiny)

Goal: one tiny app that connects Compose UI + state + ViewModel + navigation + Room.

17.1 16.1 Feature outline

- Screen A: task list
- Screen B: add task
- Persist tasks locally with Room
- ViewModel exposes `StateFlow<TasksUiState>`

17.2 16.2 Core pieces

17.2.1 Entity + DAO

```
@Entity(tableName = "tasks")
data class TaskEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val title: String,
    val done: Boolean = false
)

@Dao
interface TaskDao {
    @Query("SELECT * FROM tasks ORDER BY id DESC")
    fun observeAll(): Flow<List<TaskEntity>>

    @Insert
    suspend fun insert(task: TaskEntity)
}
```

17.2.2 Repository

```
class TaskRepository(private val dao: TaskDao) {
    fun observeTasks(): Flow<List<Task>> =
        dao.observeAll().map { rows -> rows.map { Task(it.id, it.title, it.done) } }

    suspend fun addTask(title: String) {
        dao.insert(TaskEntity(title = title.trim()))
    }
}
```

17.2.3 ViewModel

```
class TaskViewModel(private val repo: TaskRepository) : ViewModel() {
    private val _uiState = MutableStateFlow(TasksUiState(isLoading = true))
    val uiState: StateFlow<TasksUiState> = _uiState

    init {
        viewModelScope.launch {
            repo.observeTasks().collect { tasks ->
                _uiState.value = TasksUiState(items = tasks, isLoading = false)
            }
        }
    }
}
```

```

    }

    fun addTask(title: String) {
        viewModelScope.launch {
            if (title.isBlank()) return@launch
            repo.addTask(title)
        }
    }
}

```

17.2.4 Navigation + screens

```

@Composable
fun TaskApp(vm: TaskViewModel) {
    val state by vm.uiState.collectAsStateWithLifecycle()
    val nav = rememberNavController()

    NavHost(nav, startDestination = "list") {
        composable("list") {
            TaskListScreen(
                state = state,
                onAddClick = { nav.navigate("add") }
            )
        }
        composable("add") {
            AddTaskScreen(
                onSave = { title ->
                    vm.addTask(title)
                    nav.popBackStack()
                },
                onBack = { nav.popBackStack() }
            )
        }
    }
}

```

17.3 16.3 Why this sample matters

It demonstrates modern Android defaults without overwhelming ceremony:

- Single activity
- Compose-first UI
- State-driven rendering
- ViewModel + repository separation
- Navigation Compose
- Room persistence

18 Troubleshooting quick section

- **App compiles but blank screen:** confirm `setContent { ... }` actually calls your root composable.
- **Button clicks do nothing:** verify callback wiring (`onClick` passed correctly).
- **State not updating:** state may not be `mutableStateOf`/`StateFlow`, or UI not collecting it.
- **Room query not updating UI:** DAO must return `Flow` and UI must collect.
- **Navigation crash:** route argument missing or wrong type.

19 Quick references (cheat sheets)

19.1 Kotlin mini-cheat

```

val x = 1           // immutable
var y = 2           // mutable
val s: String? = null

```

```
val len = s?.length ?: 0
```

19.2 Compose state mini-cheat

```
var text by rememberSaveable { mutableStateOf("") }  
TextField(value = text, onValueChange = { text = it })
```

19.3 Coroutine + Flow mini-cheat

```
viewModelScope.launch {  
    repository.observe().collect { value ->  
        // update ui state  
    }  
}
```

19.4 Room mini-cheat

```
@Dao  
interface ExampleDao {  
    @Query("SELECT * FROM item")  
    fun observeItems(): Flow<List<ItemEntity>>  
}
```

20 Glossary

- **AAB**: Android App Bundle, preferred Play upload format.
- **Activity**: Android component hosting app UI entry point.
- **Composable**: function that describes part of UI in Compose.
- **Coroutine**: lightweight async execution unit in Kotlin.
- **DataStore**: Jetpack key-value persistence API.
- **Flow**: cold asynchronous stream of values.
- **Gradle**: build automation system used by Android projects.
- **Manifest**: app declaration file (AndroidManifest.xml).
- **Material 3**: modern Android design system/components.
- **Recomposition**: Compose re-running UI functions when state changes.
- **Repository**: abstraction coordinating data access.
- **Room**: Jetpack SQLite abstraction layer.
- **State hoisting**: moving state up and passing it down as parameters.
- **ViewModel**: lifecycle-aware holder for UI state/logic.
- **WorkManager**: API for guaranteed deferrable background work.

21 Index (A-Z)

A: Accessibility, Activities, ADB, AAB
B: Breakpoints, Build errors
C: Composable, Configuration changes, Context, Coroutines
D: DataStore, Dependencies, Debugging, DI (mentioned conceptually)
F: Flow
G: Gradle, Glossary
I: INTERNET permission, Index
L: Lifecycle, Logcat
M: Manifest, Material 3, Mutable state, MVVM framing
N: Navigation Compose, Null safety
P: Permissions, Play Store, Previews, Python bridge
R: Recomposition, Repository, Resources, Room
S: State hoisting, Security checklist, Signing
T: Testing, Troubleshooting, Themes
U: Unidirectional data flow, UI state
V: ViewModel, Versioning
W: WorkManager

22 References (official docs first)

- Android Developers: Architecture recommendations
<https://developer.android.com/topic/architecture/recommendations>
- Android Developers: Compose documentation
<https://developer.android.com/jetpack/compose>
- Android Developers: Navigation Compose
<https://developer.android.com/jetpack/compose/navigation>
- Android Developers: Room
<https://developer.android.com/training/data-storage/room>
- Android Developers: DataStore
<https://developer.android.com/topic/libraries/architecture/datastore>
- Android Developers: WorkManager
<https://developer.android.com/topic/libraries/architecture/workmanager>
- Android Developers: Runtime permissions
<https://developer.android.com/training/permissions/requesting>
- Kotlin docs: language guide
<https://kotlinlang.org/docs/home.html>

Notes: snippets are intentionally compact and may omit imports; adapt package names and dependency versions to your project template.